CSE 4502: Operating Systems Lab

Project Description

Extending the mCertiKOS kernel

No.	Project
1	UNIX Shell
2	Scheduler
3	Advanced Synchronization – File Sharing
4	Advanced Synchronization – Wait for Multiple Objects
5	Advanced Memory Allocation
6	Advanced Synchronization – User-level Mutex
7	Publish/Subscribe IPC

S. M. Sabit Bananee

Department of Computer Science and Engineering

Islamic University of Technology

BSc Engg. CSE ▪ Winter 2025–26

Contents

1 Introduction 2

2 Getting Started 3

3 Timeline and Deliverables 4

4 Evaluation 6

Projects

P1 UNIX Shell 7

P2 Scheduler 10

P3 Advanced Synchronization – File Sharing 13

P4 Advanced Synchronization – Wait for Multiple Objects 16

P5 Advanced Memory Allocation 19

P6 Advanced Synchronization – User-level Mutex 22

P7 Publish/Subscribe IPC 25

1 Introduction

In the seven lab experiments you built the core of the mCertiKOS kernel:

- physical and virtual memory management;
- threads and scheduling;
- user processes and system calls;
- multiprocessor synchronization;
- a file system.

In the project, your team *extends* that kernel with a new operating system feature of your choice. You design it, implement it, test it, and demonstrate it with user programs.

Unlike the experiments, the projects come with no skeleton code and no automatic tests. Each project description states *what* the feature must do: its requirements and the interface it must offer. It gives hints about *how*, but the design decisions are yours, and you must explain and justify them. Your grade depends on the correctness and completeness of your implementation, the quality of your design and testing, and how well you document and present your work.

List of projects

No.	Project	Main operating system topics
1	UNIX Shell (p. 7)	file descriptors, pipes, process creation and termination, redirection
2	Scheduler (p. 10)	dynamic-priority scheduling, multiprocessor load balancing, benchmarking
3	File Sharing (p. 13)	readers–writer locks, file locking, blocking and non-blocking waits
4	Wait for Multiple Objects (p. 16)	event notification, sleep/wakeup on many channels, timeouts
5	Advanced Memory Allocation (p. 19)	buddy allocation, contiguous frames, super-pages, the heap (`brk`)
6	User-level Mutex (p. 22)	shared memory, futexes, user-level synchronization primitives
7	Publish/Subscribe IPC (p. 25)	message passing, topic-based communication, bounded queues

Rules

1. **Teams.** Projects are done in teams of four or five students. Every member must contribute code and must be able to explain the whole design.
2. **Choice.** Each team chooses *one* project in Week 10. The course teacher may limit how many teams choose the same project, so register your choice early.
3. **Base code.** Every project starts from your own kernel after Experiment 7 and is built with `make LAB=8` (Section 2). A team starts from the kernel of one of its members.

4. **Keep the experiments working.** Your changes must not break the experiments: `make LAB=n TEST=1` for $n = 1, \ldots, 6$ and `fstest` in `LAB=7` must still pass. The only exceptions are changes the project requires explicitly; document them in your report.
5. **Academic integrity.** You may read the references and discuss ideas with other teams, but all code must be written by your team. Copying code from other teams, previous years or the Internet is academic misconduct. Code taken from the references must be adapted, understood, and cited in your report.

2 Getting Started

2.1 The project build: LAB=8

The project is built and run with

```
make LAB=8              # build
make qemu-nox           # run (exit with Ctrl-a x)
```

With `LAB=8` the kernel boots exactly as in Experiment 7: all layers are initialized, the file system is mounted, and the `idle` process starts. Instead of the file system test, `idle` runs the code in the `LAB == 8` block of user/idle/idle.c. Spawn your demonstration programs there.

> **Note.** In `LAB=7` and `LAB=8` the kernel runs all processes on CPU 0. If your project needs the other CPUs (Projects 2, 3, 6 and 7), start them as the `LAB == 6` code in kern/init/init.c does, and give each CPU a process to run.

2.2 Host systems and measurements

Build the project in the same environment you used for the experiments: the archive for your host system (Ubuntu/Debian, Fedora, macOS or Windows with WSL 2) and Chapter 2 of the lab handout. The code is the same for every host.

> **Note.** Timing results depend on the host. With KVM (Linux, and usually WSL 2 on Windows 11) `rdtsc()` counts real processor cycles. Without KVM (macOS, or a PC without virtualization) QEMU emulates the processor with `-icount`, so `rdtsc()` counts emulated time and absolute numbers differ by a large factor. Projects that report measurements (Projects 2 and 6) should run every configuration they compare on the same machine and setup, and state in the report which host and whether KVM was used.

2.3 Adding a user program

User programs are compiled into the kernel image and started with `spawn(elf_id, quota)`. To add a program named `myprog`:

1. Create user/myprog/myprog.c with a `main` function, and user/myprog/Makefile.inc by copying user/fork_test/Makefile.inc and replacing every `fork_test` with `myprog` and `FORK_TEST` with `MYPROG`.
2. In user/Makefile.inc, add `include $(USER_DIR)/myprog/Makefile.inc` and add `myprog` to the `user:` target.

3. In kern/trap/TSyscall/TSyscall.c, declare the start of the program's image and add a new `case` with a free `elf_id` (6, 7, ...) to `sys_spawn`:

```
extern uint8_t _binary___obj_user_myprog_myprog_start[];
...
case 6:
    elf_addr = _binary___obj_user_myprog_myprog_start;
    break;
```

2.4 Adding a system call

1. Add a number before `MAX_SYSCALL_NR` in `enum __syscall_nr`, and new error numbers (if any) before `MAX_ERROR_NR`, in kern/lib/syscall.h.
2. Implement the handler, e.g. `void sys_myfunc(tf_t *tf)`, in the layer where it belongs, and declare it in that layer's export.h. Read the arguments with `syscall_get_arg2(tf)`, ..., and return results with `syscall_set_errno` and `syscall_set_retval1`, ...
3. Add a `case` for the new number in `syscall_dispatch` (kern/trap/TDispatch/TDispatch.c) and declare the handler in TDispatch/import.h.
4. Add a user wrapper in user/include/syscall.h, following `sys_spawn`: put the number in `eax` and the arguments in `ebx`, `ecx`, `edx`, `esi`, `edi`, then execute `int $48`.

Caution. A system call receives *user* addresses. Never dereference them in the kernel. Check that they lie in [`VM_USERLO`, `VM_USERHI`) and copy data with `pt_copyin` and `pt_copyout`.

2.5 What the kernel provides

- **Memory:** `palloc`/`pfree` (single pages), containers with quotas, `map_page`, `alloc_page` (on page faults).
- **Threads:** per-CPU ready queues, `thread_yield`, timer preemption (`sched_update`), `thread_sleep`/`thread_wakeup`, spinlocks.
- **Processes:** `proc_create`, the system calls `spawn`, `yield`, `produce`/`consume` (bounded buffer).
- **File system:** `open`, `read`, `write`, `close`, `fstat`, `mkdir`, `chdir`, `link`, `unlink`, with per-process file descriptor tables.
- **Not provided:** process termination (`exit`/`wait`), user-level threads, shared memory, console input for user programs, and `exec` from the file system. Projects that need these list them as requirements.

Tip. Use Git: create a branch `project` from your Experiment 7 commit and commit after every working step. Measure time with `rdtsc()` (available to user programs in user/include/x86.h) when a project asks for benchmarks.

3 Timeline and Deliverables

3.1 Timeline

When	Milestone
Week 10	Teams formed and projects chosen; discussion of the requirements with the course teacher.
Week 11, Week 12, Week 13	**Proposal** (1–2 pages) submitted at the start of the lab, followed by a **progress review**: a working part of the design must be shown.
Week 14	**Demonstration and presentation** in the lab; final submission of the code and the report.

3.2 Proposal (Week 11, Week 12, Week 13)

The proposal is a short design document (1–2 pages) containing:

1. the team members and the chosen project;
2. the design: new kernel data structures, new or changed system calls (with their exact signatures), and the layers or files you will change;
3. how you handle concurrency: which locks protect which data, and where threads sleep and wake up;
4. the test and demonstration programs you will write;
5. the division of work between the members.

3.3 Final submission (Week 14)

Submit one archive (or a link to a private Git repository) per team containing:

1. **The complete source tree** that builds with `make LAB=8`, including your user test and demonstration programs. Remove the build output first (`make clean`).
2. **A report** (PROJECT.md or a PDF, about 4–8 pages) with the following sections:
   - *Overview*: what you implemented and what you did not;
   - *Design*: data structures, algorithms, system call interface, synchronization;
   - *Implementation*: the files and functions you added or changed;
   - *Testing*: the test programs and their results (and benchmarks where required);
   - *Limitations and known bugs*;
   - *Contributions*: who did what;
   - *References* you used.
3. **Presentation slides** (about 8–10 minutes per team).

3.4 Demonstration (Week 14)

Each team presents its design and demonstrates the project live from a clean build: `make clean && make LAB=8 && make qemu-nox`. Each project description lists what the demonstration must show. Prepare your demonstration programs so that their output is easy to follow. After the demonstration, every member answers individual questions about the design and the code.

4 Evaluation

The project is worth 30% of the course marks. Each team's project is marked out of 100 using the rubric below. A member's individual mark can be lowered if the viva shows that the member did not contribute or cannot explain the work.

Component	Criteria	Marks
Proposal	clear design, realistic plan, sensible system call interface	10
Progress review	a working part of the design is shown in Week 11, Week 12, Week 13	10
Functionality	all mandatory requirements of the project work correctly, including error cases and concurrent use	40
Design and code quality	sound data structures and synchronization, layered and readable code, no busy-waiting where sleeping is required	10
Testing and demonstration	meaningful test programs; the demonstration shows every requirement; benchmarks where required	15
Report and presentation	complete and accurate report; clear presentation	10
Viva	each member can explain the design and the code	5
	Total	**100**

Optional extensions listed in a project can earn up to 10 bonus marks, up to the maximum of 100. They count only after all mandatory requirements work.

Project 1: UNIX Shell

Topics	file descriptors, console I/O, pipes, process creation and termination, I/O redirection
Builds on	Experiments 5 (processes, system calls), 6 (sleep/wakeup) and 7 (file system)
Demonstration	an interactive shell running `cat file | rot13 > file.encrypted`

P1.1 Overview

A shell is the program that reads commands from the user and runs them as processes. The UNIX shell made a few simple but powerful design decisions:

- every program reads its input from file descriptor 0 and writes its output to file descriptor 1;
- a file descriptor can refer to the console, a file or a pipe, and the program cannot tell the difference;
- the shell, not the program, decides where these descriptors point.

These decisions make redirection (`>`, `<`) and pipelines (`|`) possible without changing any program.

mCertiKOS has no shell yet. User programs print with the `puts` system call, cannot read the keyboard, cannot end (`exit`), and file descriptors exist only for files. In this project you unify console I/O, inter-process communication and file I/O behind file descriptors, and build a shell on top of them.

P1.2 Learning objectives

- Design a uniform file descriptor abstraction over different kinds of objects.
- Implement pipes as a kernel bounded buffer with blocking reads and writes and end-of-file.
- Implement process termination and waiting.
- Understand how a shell implements redirection and pipelines with descriptors.

P1.3 Starting point in mCertiKOS

- kern/fs/file.h: `struct file` already has the type `FD_PIPE`, which is not used yet. Each process has `NOFILE` open files in its TCB (`tcb_get_openfiles`).
- kern/fs/sysfile.c: `sys_read`, `sys_write`, `sys_open`, `sys_close` (your Experiment 7 code).
- kern/dev/console.c: `cons_getc()` returns a character from the keyboard or serial port (0 if none); the kernel monitor uses it through `readline`.
- kern/trap/TSyscall/TSyscall.c: `sys_spawn`; user/lib/entry.S: what happens when `main` returns.

P1.4 Requirements

R1 **Console descriptors.** Add console input and output as a new file type. Every process starts with descriptor 0 as its input and descriptor 1 as its output, both connected to the

console by default. `read(0, ...)` blocks until input is available and must not busy-wait. `write(1, ...)` prints.

R2 **Uniform output.** `printf` in the user library writes through `write(1, ...)`. The `puts` system call is no longer used by user programs.

R3 **Pipes.** Add a system call `pipe(int fd[2])` that creates a pipe with a bounded kernel buffer. `fd[0]` is the read end and `fd[1]` the write end. A reader sleeps while the pipe is empty, and a writer sleeps while it is full. When all write ends are closed, `read` returns 0 (end of file). When all read ends are closed, `write` returns an error.

R4 **Spawning with I/O channels.** Extend process creation so that the parent chooses the child's input and output descriptors, for example `spawn(elf_id, quota, in_fd, out_fd)`. The child receives them as its descriptors 0 and 1, sharing the open file with the parent (reference counting, `file_dup`).

R5 **Arguments.** A spawned program receives command-line arguments as `main(int argc, char **argv)`, for example by copying them onto the child's user stack before it starts.

R6 **Termination.** Add `exit(status)` and `wait(pid)`. `exit` closes all descriptors of the process and wakes up a waiting parent. Returning from `main` calls `exit`. A process that exits never runs again. Releasing its memory and its id is an optional extension.

R7 **Commands.** Implement at least `echo`, `cat`, `ls`, `wc` and `rot13` as *separate* user programs that read descriptor 0 (or the files named in their arguments) and write descriptor 1. The shell maps command names to programs.

R8 **The shell.** An interactive shell with a prompt that supports output redirection (`> file`), input redirection (`< file`) and pipelines of at least two commands (`a | b`), and combinations such as `cat file | rot13 > file.encrypted`. The shell waits for every command it starts.

Optional extensions

- Pipelines of any length; background jobs (`&`); built-in commands `cd` and `mkdir`.
- `exec`: load programs from ELF files in the file system instead of compiling them into the kernel.
- Freeing all memory and the process id on `exit`, so that processes can be created repeatedly.

P1.5 Suggested interface

```
int   pipe(int fd[2]);                          /* 0 on success, -1 on error
     */
pid_t spawn(unsigned int elf_id, unsigned int quota,
            int in_fd, int out_fd, char *argv[]); /* child pid, or -1
      */
void  exit(int status);                         /* never returns
     */
int   wait(pid_t pid, int *status);             /* 0 on success, -1 if not a child
     */
int   read(int fd, char *buf, int n);           /* existing: now also console and
    pipe */
int   write(int fd, char *buf, int n);
```

P1.6 Implementation guidance

1. Implement the console file type and connect descriptors 0 and 1 in `proc_create`. For input, the console interrupt (or a polling loop that yields) can wake up readers sleeping on a console channel.
2. Change `printf` to use `write(1, ...)`. Test it with the existing programs.
3. Implement `pipe` with a circular buffer, a spinlock, and two sleep channels, as in the bounded buffer of Experiment 6. Count readers and writers for end-of-file.
4. Extend `spawn` with descriptors and arguments, then `exit` and `wait`.
5. Write the commands, then the shell: parse the line into commands, create pipes, spawn the commands with the right descriptors, close the parent's copies, and wait.

> **Caution.** The most common pipeline bug is forgetting to close a write end. Then the reader never sees end-of-file and the pipeline hangs.

P1.7 Demonstration

```
$ echo hello operating systems > a.txt
$ cat a.txt
$ cat a.txt | rot13 > b.txt
$ cat b.txt
$ cat b.txt | rot13 | wc
$ ls
$ wc < a.txt
```

Show also that a reader blocks (without consuming CPU) on an empty pipe, and that it receives end-of-file when the writer exits.

P1.8 References

1. R. Cox, F. Kaashoek, R. Morris, *xv6: a simple, Unix-like teaching operating system*, Chapter 0 (operating system interfaces: I/O, pipes, the shell).
2. R. H. and A. C. Arpaci-Dusseau, *Interlude: Process API*, Operating Systems: Three Easy Pieces.
3. Linux manual pages: pipe(2), dup(2).

Project 2: Scheduler

Topics	CPU scheduling, dynamic priorities, starvation, multiprocessor load balancing, performance evaluation
Builds on	Experiments 4 (threads and scheduling), 5 (timer preemption) and 6 (multiprocessor)
Demonstration	CPU-bound and interactive processes on 4 CPUs, with measured results

P2.1 Overview

The scheduler you built in the experiments is round-robin: every ready thread waits its turn in the ready queue of its CPU, whatever it does. Real schedulers distinguish between processes:

- an *interactive* process that runs briefly and then waits should get the CPU quickly;
- a *CPU-bound* process can wait longer.

They do this with *dynamic priorities*: the priority of a process goes down while it uses the CPU and up while it waits. On a multiprocessor, the scheduler must also keep all CPUs busy. If one CPU has many ready processes and another has none, it *balances the load* by moving processes between CPUs.

In this project you replace the round-robin scheduler with a dynamic-priority scheduler with load balancing, and evaluate it with benchmarks. The Plan 9 scheduler is a clear and simple model.

P2.2 Learning objectives

- Design and implement a dynamic-priority scheduling algorithm that avoids starvation.
- Implement load balancing across per-CPU ready queues safely.
- Measure and compare scheduling policies with benchmarks.

P2.3 Starting point in mCertiKOS

- kern/thread/PThread/PThread.c: `thread_spawn`, `thread_yield`, `sched_update` (called on every timer interrupt), `sched_lk`.
- kern/thread/PTQueueInit/: ready queue `NUM_IDS + cpu` for each CPU.
- kern/lib/thread.h: `SCHED_SLICE`; kern/dev/lapic.h: `LAPIC_TIMER_INTR_FREQ` (1000 timer interrupts per second).
- kern/init/init.c: the `LAB == 6` code starts all CPUs.

P2.4 Requirements

R1 **Priority levels.** Support at least 8 priority levels, each with its own ready queue on every CPU (or an equivalent structure). The scheduler always runs a ready process of the highest priority. Processes of equal priority are scheduled round-robin.

R2 **Dynamic priority.** A process that uses its whole time slice loses priority. A process that gives up the CPU early (by yielding or sleeping) keeps or gains priority. Higher priority levels may use shorter time slices. Describe your exact rules.

R3 **No starvation.** A ready process must eventually run, for example through a periodic priority boost or aging.

R4 **Base priority.** Add a system call `setpriority(pid, prio)` (like `nice`) that sets the base priority from which the dynamic priority starts.

R5 **Load balancing.** Run processes on all CPUs. When a CPU has no ready process, or the queue lengths differ by more than a threshold, move ready processes to less loaded CPUs. Use correct locking: a process must never be in two queues or run on two CPUs at once.

R6 **Statistics.** Add a system call `getpinfo(pid, struct pinfo *)` returning at least the priority, the CPU, the number of times the process was scheduled, and the CPU time it used.

R7 **Benchmarks.** Compare round-robin and your scheduler on at least two workloads, for example:

- CPU-bound processes only;
- CPU-bound processes mixed with interactive processes that compute briefly and yield.

Report the turnaround time, the response time of the interactive processes, and the CPU utilization per CPU, measured with `rdtsc()`.

Optional extensions

- Processor affinity: prefer the CPU on which a process ran last.
- A second policy (e.g. lottery or stride scheduling), selectable at build time, included in the benchmarks.

P2.5 Suggested interface

```
struct pinfo { unsigned int prio, base_prio, cpu, nscheduled; unsigned long long
    cpu_ticks; };
int setpriority(pid_t pid, unsigned int prio);   /* 0 = highest priority */
int getpinfo(pid_t pid, struct pinfo *info);
```

P2.6 Implementation guidance

1. Record per process: the current and base priority, the ticks used in the current slice, and the total ticks.
2. In `sched_update`, count ticks and decide whether to preempt and lower the priority. In `thread_yield` and `thread_sleep`, apply the rule for early yielding.
3. Pick the next thread from the highest non-empty queue of the current CPU.
4. For load balancing, an idle CPU can *steal* a thread from the busiest CPU. Lock both queues in a fixed order (e.g. by CPU number) to avoid deadlock.
5. Write the benchmark programs first, so that you can measure round-robin before changing it.

P2.7 Demonstration

Run a mixed workload on 4 CPUs and show:

- the priorities changing over time (with `getpinfo`);
- interactive processes getting short response times;
- processes migrating to idle CPUs.

Present your benchmark table and explain the results.

P2.8 References

1. Plan 9 from Bell Labs, scheduler source code sys/src/9/port/proc.c (functions `sched`, `runproc`, `ready`, `updatecpu`).
2. R. Pike et al., *Process Sleep and Wakeup on a Shared-memory Multiprocessor*, Plan 9 documentation.
3. R. H. and A. C. Arpaci-Dusseau, Operating Systems: Three Easy Pieces: *Multi-level Feedback Queue*, *Multiprocessor Scheduling*, *Proportional Share*.
4. The Linux kernel documentation, *CFS Scheduler*.

Project 3: Advanced Synchronization – File Sharing

Topics	readers–writer locks, file locking, blocking and non-blocking synchronization, starvation
Builds on	Experiments 6 (sleep/wakeup, multiprocessor) and 7 (file system)
Demonstration	several processes on different CPUs updating a shared file correctly with `flock`

P3.1 Overview

When several processes use the same file, each individual `read` or `write` is atomic in mCertiKOS, but a *sequence* of operations is not. Suppose two processes each read a counter from a file, add one, and write it back. Both can read the same value, and one increment is lost.

UNIX systems solve this with *advisory file locks*. In Linux, `flock(fd, op)` places a lock on an open file in one of two modes:

- **Shared** (`LOCK_SH`): any number of processes may hold a shared lock at the same time, for example readers.
- **Exclusive** (`LOCK_EX`): only one process may hold the lock, and no shared locks may be held at the same time; for example a writer.

A process that requests a conflicting lock waits until the lock becomes available, unless it asks not to wait (`LOCK_NB`). `LOCK_UN` releases the lock, and closing the file releases it too. This is the classic readers–writers problem, applied to files.

P3.2 Learning objectives

- Implement a readers–writer lock in the kernel with sleep and wakeup.
- Decide and implement a fairness policy that prevents starvation.
- Integrate locks with the life cycle of open files.

P3.3 Starting point in mCertiKOS

- kern/fs/inode.h: `struct inode`, one in-memory object per file, shared by all processes that open the file.
- kern/fs/file.c, kern/fs/sysfile.c: open files and the file system calls (Experiment 7), including `sys_close`.
- kern/thread/PThread/PThread.c: `thread_sleep`, `thread_wakeup` (Experiment 6).

P3.4 Requirements

R1 **System call.** Implement `flock(fd, op)` with `op` one of `LOCK_SH`, `LOCK_EX`, `LOCK_UN`, optionally combined with `LOCK_NB`. It returns 0 on success and -1 on error.

R2 **Semantics.** A shared lock is granted when no exclusive lock is held. An exclusive lock is granted when no other lock is held. Locks belong to the file (the inode), so they work between different processes that opened the same file.

R3 **Blocking.** A request that cannot be granted sleeps until it can (no busy-waiting). With `LOCK_NB`, it returns immediately with a new error number `E_WOULDBLOCK`.

R4 **Conversion.** A process holding a lock may request the other mode. As in Linux, the conversion need not be atomic: the old lock may be released before the new one is acquired.

R5 **Release.** `LOCK_UN` and `close` release the lock of the process and wake up the waiters.

R6 **Fairness.** Waiting writers must not starve while readers keep arriving. Implement and document a policy, for example: once a writer waits, new readers wait too.

R7 **Errors.** Invalid descriptors or operations return an error.

Optional extensions

- Byte-range locks (`fcntl`-style), where processes lock disjoint parts of a file independently.
- Deadlock detection between two files locked in opposite orders, returning an error (`EDEADLK`).

P3.5 Suggested interface

```c
#define LOCK_SH 1      /* shared lock          */
#define LOCK_EX 2      /* exclusive lock       */
#define LOCK_NB 4      /* do not block         */
#define LOCK_UN 8      /* unlock               */
int flock(int fd, int op);

/* usage */
if (flock(fd, LOCK_EX) == -1) { /* error */ }
/* ... read, modify and write the file ... */
flock(fd, LOCK_UN);
```

P3.6 Implementation guidance

1. Add to `struct inode` a spinlock, the number of shared holders, the exclusive holder, and the number of waiting writers.
2. Record in each process which files it holds locks on, so that `close` can release them.
3. Implement acquire and release with `thread_sleep` and `thread_wakeup`, re-checking the condition in a loop after every wakeup.

P3.7 Demonstration

1. **Lost updates:** 4 processes on different CPUs each increment a counter stored in a file 1000 times. Show the wrong result without locks and the correct result (4000) with `LOCK_EX`.
2. **Readers and writers:** several readers hold `LOCK_SH` at the same time, while a writer waits until they release. Show the order of events.
3. **Non-blocking:** `LOCK_NB` returns `E_WOULDBLOCK` while another process holds the lock.
4. **Fairness:** a writer is not starved by a continuous stream of readers.

P3.8 References

1. Linux manual page flock(2).
2. V. Gaydov, *File locking in Linux*, 2016.
3. R. H. and A. C. Arpaci-Dusseau, *Semaphores* (Section 31.5, reader–writer locks), Operating Systems: Three Easy Pieces.

Project 4: Advanced Synchronization – Wait for Multiple Objects

Topics	event notification, sleep/wakeup on multiple channels, polling vs. sleeping, timeouts
Builds on	Experiments 5 (system calls), 6 (sleep/wakeup, bounded buffer)
Demonstration	a server process waiting on several channels and a timer at once

P4.1 Overview

A thread often needs to wait for *any* of several events. A server, for example, may wait for:

- messages on several communication channels;
- keyboard input;
- the completion of a disk request;
- a timeout.

With only blocking calls such as `consume`, it can wait for one event at a time, and polling every source in a loop wastes the CPU. Operating systems therefore offer one system call that waits on a whole set of notification sources:

- `select` and `poll` in UNIX;
- `epoll` in Linux;
- `kqueue` in BSD and macOS.

`epoll` works in three steps:

1. create an event object (`epoll_create`);
2. register the sources to watch (`epoll_ctl`);
3. wait until one or more of them are ready (`epoll_wait`).

In this project you design and implement a similar mechanism for mCertiKOS.

P4.2 Learning objectives

- Design an abstraction for notification sources and event sets.
- Make one thread wait on many sleep channels without losing wakeups.
- Implement polling, blocking and timeout modes correctly on a multiprocessor.

P4.3 Starting point in mCertiKOS

- kern/thread/PThread/PThread.c: `thread_sleep(chan, lk)` sleeps on *one* channel.
- kern/trap/TSyscall/bbuf.c: one bounded buffer used by `produce`/`consume`.
- kern/trap/TTrapHandler/TTrapHandler.c: the timer interrupt (1000 per second) can drive timers.

P4.4 Requirements

R1 **Notification sources.** Support at least three kinds of sources:

(a) *message channels*: generalize the bounded buffer to several channels created at run time, with `chan_create`, `chan_send` and `chan_recv`;

(b) *timers*: one-shot or periodic, in milliseconds;

(c) *one more source of your choice*, such as console input or a file.

R2 **Event objects.** `wq_create()` creates an event object and returns a handle. `wq_ctl(handle, op, source, events)` adds, modifies or removes a source and the events of interest (e.g. *readable*, *writable*, *expired*).

R3 **Waiting.** `wq_wait(handle, ready[], max, timeout)` returns the ready sources (at most `max`):

- `timeout = 0`: **polling**, return immediately;
- `timeout < 0`: **sleeping** until at least one source is ready, without consuming CPU;
- `timeout > 0`: sleep for at most that many milliseconds.

R4 **No lost wakeups.** An event that happens between checking the sources and going to sleep must still wake the waiter. Several processes on different CPUs may use the same sources.

R5 **Level-triggered semantics.** A source is reported as long as it is ready, for example while a channel still contains messages.

R6 **Clean-up.** Removing a source, or destroying a channel while it is registered, must not leave dangling references.

Optional extensions

- Edge-triggered mode (report a source only when it *becomes* ready).
- Waking up only the waiters that registered for the source that became ready (no scanning of all event objects), with an evaluation of the cost.

P4.5 Suggested interface

```c
#define EV_READABLE 1
#define EV_WRITABLE 2
#define EV_EXPIRED  4
#define WQ_ADD 1
#define WQ_MOD 2
#define WQ_DEL 3
struct wq_event { int source; int events; };

int chan_create(int capacity);
int chan_send(int chan, int msg);              /* blocks while full  */
int chan_recv(int chan, int *msg);             /* blocks while empty */
int timer_create(int ms, int periodic);

int wq_create(void);
int wq_ctl(int wq, int op, int source, int events);
int wq_wait(int wq, struct wq_event *ready, int max, int timeout_ms);
```

P4.6 Implementation guidance

1. Give every source a list of the event objects watching it. When the source changes state (a message arrives, a timer expires), it notifies those event objects.
2. An event object has its own lock, a ready list and a sleep channel. `wq_wait` sleeps on that channel, so a single sleep covers all sources.
3. Hold the event object's lock while checking the ready list and calling `thread_sleep`, so that no notification is lost.
4. For timeouts, register a kernel timer that wakes up the waiter when it expires.

P4.7 Demonstration

- A server process waits with one `wq_wait` on three channels fed by three producer processes on different CPUs, and on a periodic timer. It prints every event as it is handled.
- Show the difference between polling (`timeout = 0`) and sleeping (`timeout < 0`), for example by counting how often each mode returns with no event.
- Show that a timeout expires correctly when no source is ready.

P4.8 References

1. Linux manual pages epoll(7) and poll(2).
2. C. Sridharan, *The method to epoll's madness*, 2016.
3. NetBSD, *kqueue tutorial*.
4. J. Corbet, *The kevent interface*, LWN.net, 2006.
5. D. Kegel, *The C10K problem*.

Project 5: Advanced Memory Allocation

Topics	physical memory allocation, buddy system, fragmentation, super-pages (PSE), the process heap
Builds on	Experiments 1 (physical memory), 2 and 3 (page tables, virtual memory) and 5 (page faults)
Demonstration	a program allocating contiguous memory and super-pages with `brk`, with measurements

P5.1 Overview

Memory allocation in mCertiKOS is primitive. `palloc` returns one 4 KB page at a time, found by scanning the allocation table. A user process gets memory one page at a time, on a page fault. Many uses need more:

- Devices that use DMA need buffers that are *physically contiguous*.
- Large memory regions can be mapped with *super-pages*. On 32-bit x86 with Page Size Extension (PSE), one page directory entry maps 4 MB. This saves page tables and TLB entries, and therefore TLB misses.
- Programs manage their heap with `brk`, which moves the end of the data segment up (to allocate) and down (to free).

In this project you implement a buddy allocator for contiguous physical memory, super-page mappings, and the `brk` system call. The allocator must handle fragmentation robustly.

P5.2 Learning objectives

- Implement and analyze the buddy system, including splitting and coalescing.
- Use x86 4 MB pages and measure their effect on TLB behaviour.
- Manage the address space of a process (heap growth and shrinking) together with quotas.

P5.3 Starting point in mCertiKOS

- kern/pmm/MATIntro, MATOp: the allocation table and `palloc`/`pfree` (Experiment 1); kern/pmm/MContainer: quotas (Experiment 2).
- kern/vmm/: page directory and page table functions, `map_page`, `alloc_page` (Experiments 2 and 3).
- kern/lib/x86.h: `PTE_PS` (4 MB page bit in a page directory entry). `CR4.PSE` must be enabled for it to take effect.

P5.4 Requirements

R1 **Buddy allocator.** Manage the normal physical pages with a buddy allocator of orders 0 (4 KB) to at least 10 (4 MB). `palloc_order(order)` returns 2^{order} physically contiguous, naturally aligned pages. Freed blocks are coalesced with their buddies. `palloc` and `pfree` keep working, as order 0, so that all experiments still pass.

R2 **Contiguous allocation for processes.** A process can request n physically contiguous pages, mapped at consecutive virtual addresses. A system call `va2pa(va)` (for testing) shows that the physical addresses are consecutive.

R3 **Super-pages.** Enable PSE and map 4 MB-aligned regions of a process with single page directory entries (`PTE_PS`) backed by order-10 blocks.

R4 **`brk`.** Every process has a heap that starts at a fixed, documented virtual address. `brk(addr, flags)` moves the end of the heap to `addr`:

- growing maps new memory, as ordinary pages, contiguous pages, or super-pages depending on `flags`;
- shrinking unmaps the memory and frees the pages (and page tables that become empty).

It returns the new end of the heap, or -1 on error.

R5 **Quotas and errors.** All allocations are charged to the process's container. Requests that exceed the quota or cannot be satisfied (e.g. no free block of that order) fail cleanly.

R6 **Fragmentation.** Report the number of free blocks of each order (through a system call or a monitor command). Demonstrate that freeing memory restores large blocks.

Optional extensions

- A user-level `malloc`/`free` library on top of `brk`.
- Lazy heap growth: map ordinary heap pages on the first page fault instead of in `brk`.

P5.5 Suggested interface

```
#define BRK_PAGES   0      /* ordinary 4 KB pages             */
#define BRK_CONTIG  1      /* physically contiguous pages     */
#define BRK_SUPER   2      /* 4 MB super-pages (4 MB aligned) */
void *brk(void *addr, int flags);   /* new heap end, or (void *) -1 */
unsigned int va2pa(void *va);       /* physical address (testing)   */
int buddy_stats(unsigned int free_blocks[11]);
```

P5.6 Implementation guidance

1. Keep one free list per order and record each free block's order in the allocation table. The buddy of block i of order k is block $i \oplus 2^k$.
2. Build the free lists in `pmem_init` from the usable page ranges. Remember that the usable ranges are not aligned to large blocks.
3. For super-pages, set `CR4.PSE` at boot (with `rcr4`/`lcr4`) and write page directory entries of the form `(page_index « 12) | PTE_PS | PTE_P | PTE_W | PTE_U`. The functions that walk page tables must recognize such entries.
4. Measure with `rdtsc()`: for example, touch one word in every 4 KB page of a 16 MB region, mapped once with 4 KB pages and once with super-pages, and compare.

> **Caution.** Code that reads a page directory entry as a pointer to a page table must not do so for a 4 MB entry. Find every such place (including `pt_copyin`/`pt_copyout` and `unmap_page`).

P5.7 Demonstration

- Grow and shrink the heap with `brk` in each mode, and print the virtual and physical addresses.
- Show the free block counts per order after allocating and freeing a mix of sizes.
- Present the timing comparison between 4 KB pages and super-pages, and explain it.
- Show that all experiment tests still pass.

P5.8 References

1. Linux manual page brk(2).
2. R. H. and A. C. Arpaci-Dusseau, Operating Systems: Three Easy Pieces: *Free-Space Management* (buddy allocation), *Paging: Faster Translations (TLBs)*.
3. M. Gorman, *Understanding the Linux Virtual Memory Manager*, Chapter 6: Physical Page Allocation.
4. Intel, *Intel 64 and IA-32 Architectures Software Developer's Manual*, Volume 3A, Chapter 4 (Paging; 32-bit paging with 4 MB pages).
5. OSDev Wiki, *Paging*.

Project 6: Advanced Synchronization – User-level Mutex

Topics	shared memory, atomic operations, futexes, user-level mutexes, semaphores and condition variables
Builds on	Experiments 3 (virtual memory), 5 (system calls) and 6 (sleep/wakeup, multiprocessor)
Demonstration	processes on 4 CPUs synchronizing through shared memory with futex-based locks

P6.1 Overview

Processes that share memory need locks. A spinlock in user space wastes CPU while it waits. A lock that makes a system call on every operation is slow even when there is no contention. Linux solves this with the *futex* ("fast user-space mutex"):

- the lock is an integer in user memory, acquired with an atomic instruction;
- the kernel is involved only when a thread has to wait or has to be woken up.

A futex is essentially a kernel wait queue attached to a user address.

`futex(uaddr, op, val, ...)` supports three operations, among others:

- `FUTEX_WAIT`: if `*uaddr == val`, sleep (the check and the sleep are atomic);
- `FUTEX_WAKE`: wake up at most `val` threads waiting on `uaddr`;
- `FUTEX_CMP_REQUEUE`: wake up some waiters and move the others to another futex without waking them, which avoids the *thundering herd* problem.

mCertiKOS has no user-level threads, so in this project processes share memory. You first add shared memory, then futexes, then a library of user-level synchronization objects built on them.

P6.2 Learning objectives

- Implement shared memory between address spaces.
- Implement a futex with correct, race-free wait and wake semantics.
- Build mutexes, semaphores, condition variables and queues on futexes, and measure their cost.

P6.3 Starting point in mCertiKOS

- kern/vmm/MPTKern: `map_page` can map the same physical page into several processes.
- kern/thread/PThread/PThread.c: `thread_sleep`, `thread_wakeup`.
- user/lib/spinlock.c, user/include/x86.h: atomic `xchg` in user space; `rdtsc()` for timing.

P6.4 Requirements

R1 **Shared memory.** `shm_create(key, npages)` and `shm_attach(key, va)` map the same physical pages into every process that attaches the key. `shm_detach` removes the mapping. The pages are freed when no process uses them any more.

R2 **Futex wait.** `futex(uaddr, FUTEX_WAIT, val)` sleeps if `*uaddr == val`, and returns the error `E_AGAIN` immediately otherwise. Identify a futex by its *physical* address, so that it works across processes that map the page at different virtual addresses.

R3 **Futex wake.** `futex(uaddr, FUTEX_WAKE, n)` wakes up at most `n` waiters and returns how many it woke.

R4 **Requeue.** `futex(uaddr, FUTEX_CMP_REQUEUE, n, uaddr2, m, val3)` returns `E_AGAIN` if `*uaddr != val3`. Otherwise it wakes up at most `n` waiters and moves at most `m` of the remaining waiters to `uaddr2`.

R5 **No lost wakeups.** A wake that happens between the user's check of the lock word and the kernel's sleep must not be lost: the kernel re-checks `*uaddr` under the futex lock.

R6 **User library.** Implement in user space, on top of the futex:

- a mutex that makes no system call when uncontended;
- a counting semaphore;
- a condition variable whose broadcast uses `FUTEX_CMP_REQUEUE`;
- a FIFO bounded buffer queue.

R7 **Measurement.** Compare the cost of an uncontended lock/unlock of your futex mutex with a lock that makes a system call every time, measured with `rdtsc()`.

Optional extensions

- A timeout for `FUTEX_WAIT`.
- A readers–writer lock and a barrier in the user library.

P6.5 Suggested interface

```
#define FUTEX_WAIT        0
#define FUTEX_WAKE        1
#define FUTEX_CMP_REQUEUE 4
int futex(int *uaddr, int op, int val, int *uaddr2, int val2, int val3);

int   shm_create(int key, int npages);
void *shm_attach(int key, void *va);
int   shm_detach(void *va);

/* user library */
void mutex_lock(mutex_t *m);    void mutex_unlock(mutex_t *m);
void sem_wait(sem_t *s);        void sem_post(sem_t *s);
void cond_wait(cond_t *c, mutex_t *m);  void cond_signal(cond_t *c);  void
    cond_broadcast(cond_t *c);
```

P6.6 Implementation guidance

1. Implement shared memory first and test it with two processes that increment a shared counter under a user spinlock.
2. Keep a hash table of futex wait queues indexed by physical address, protected by a spinlock. In `FUTEX_WAIT`, read `*uaddr` (with `pt_copyin`) while holding that lock, then sleep on the queue with `thread_sleep`.

3. Implement the mutex with three states (0 unlocked, 1 locked, 2 locked with waiters), as in Drepper's paper. You will need an atomic compare-and-swap (`lock cmpxchg`) in user space.
4. Test with many processes on different CPUs and many iterations: synchronization bugs appear only under contention.

P6.7 Demonstration

- 4 processes on different CPUs increment a shared counter 10000 times each under the futex mutex: the result is exact, and contended processes sleep instead of spinning.
- A producer/consumer pair (or more) uses the bounded queue in shared memory.
- A condition variable broadcast with requeue wakes up one waiter at a time.
- Present the timing comparison.

P6.8 References

1. Linux manual pages futex(2) and futex(7).
2. U. Drepper, *Futexes Are Tricky*.
3. D. Hart, *A futex overview and update*, LWN.net, 2009.
4. E. Bendersky, *Basics of Futexes*, 2018.
5. *Thundering herd problem*, Wikipedia.
6. R. H. and A. C. Arpaci-Dusseau, Operating Systems: Three Easy Pieces: *Locks*, *Condition Variables*, *Semaphores*.

Project 7: Publish/Subscribe IPC

Topics	message passing, topic-based communication, bounded queues, non-blocking delivery
Builds on	Experiments 5 (system calls) and 6 (sleep/wakeup, bounded buffer, multiprocessor)
Demonstration	a publisher and several subscribers on different CPUs, including a slow subscriber

P7.1 Overview

Topic-based messaging is widely used in modern software, especially in robotics and embedded systems. Middleware such as ROS, OpenDDS and MQTT is built on the *publish/subscribe* (pub/sub) model:

- *publishers* send messages to a named *topic* without knowing who receives them;
- every *subscriber* of the topic receives a copy of every message.

Each subscriber has its own bounded queue. When it is full, the oldest message is dropped, so a slow subscriber can never block the publisher or the other subscribers, and the system cannot deadlock on communication.

In ROS, a publisher and a subscriber are created like this:

```
ros::Publisher chatter_pub = n.advertise<std_msgs::String>("chatter", 1000);  /*
    queue of 1000 */
chatter_pub.publish(msg);

ros::Subscriber sub = n.subscribe("chatter", 1000, chatterCallback);  /* oldest
    dropped when full */
void chatterCallback(const std_msgs::String::ConstPtr &msg) { ... }
```

ROS implements this on top of TCP/IP even when all nodes run on the same machine. Nothing prevents the operating system from offering pub/sub directly as system calls. In this project you design and implement such a subsystem for mCertiKOS.

P7.2 Learning objectives

- Design a kernel IPC interface and its naming scheme (topics).
- Implement one-to-many message delivery with per-subscriber bounded queues.
- Combine non-blocking sends with blocking receives correctly on a multiprocessor.

P7.3 Starting point in mCertiKOS

- kern/trap/TSyscall/bbuf.c: a single bounded buffer with sleep and wakeup (Experiment 6).
- kern/thread/PThread/PThread.c: `thread_sleep`, `thread_wakeup`.
- `pt_copyin`/`pt_copyout` to copy messages between user and kernel memory.

P7.4 Requirements

R1 **Topics.** Topics are identified by name (a string of up to 31 characters) and created when first advertised or subscribed. Support at least 16 topics, with several publishers and subscribers each.

R2 **Publishing.** `advertise(topic)` returns a publisher handle. `publish(handle, msg, len)` copies the message (up to a documented maximum size, e.g. 256 bytes) into the queue of every current subscriber of the topic, and *never blocks*.

R3 **Subscribing.** `subscribe(topic, queue_len)` returns a subscription handle with its own queue of `queue_len` messages. When the queue is full, a new message replaces the oldest one, and the number of dropped messages is counted.

R4 **Receiving.** `receive(handle, buf, maxlen, flags)` returns the oldest queued message, sleeping while the queue is empty, or returning `E_AGAIN` immediately with a non-blocking flag. The messages of one publisher arrive at each subscriber in the order they were published.

R5 **Clean-up.** `unsubscribe` and `unadvertise` release resources. A message published to a topic with no subscribers is discarded.

R6 **Callbacks in user space.** Provide a user library in the style of ROS: `ps_subscribe(topic, queue_len, callback)` registers a callback, and `ps_spin()` receives messages and calls the matching callbacks, possibly for several subscriptions.

R7 **Concurrency.** Publishers and subscribers on different CPUs work correctly with proper locking. Statistics (messages published, delivered, dropped) are available through a system call.

Optional extensions

- Zero-copy delivery: messages in shared memory pages instead of copies.
- "Latched" topics, where new subscribers receive the last published message immediately.

P7.5 Suggested interface

```
int advertise(const char *topic);
int publish(int pub, const void *msg, int len);          /* never blocks
    */
int subscribe(const char *topic, int queue_len);
int receive(int sub, void *buf, int maxlen, int flags);  /* flags: PS_NONBLOCK
     */
int unsubscribe(int sub);
int unadvertise(int pub);
int ps_stats(int sub, struct ps_stats *st);               /* received, dropped, ...
     */

/* user library */
int  ps_subscribe(const char *topic, int queue_len, void (*callback)(const void *msg,
     int len));
void ps_spin(void);
```

P7.6 Implementation guidance

1. A topic holds its name, a lock and a list of subscriptions. A subscription holds a circular queue of fixed-size message slots, a sleep channel, and its counters.
2. `publish` copies the message from user memory once into a kernel buffer. Then, for every subscription, under that subscription's lock, it inserts the message (dropping the oldest if full) and wakes up the receiver.
3. Lock order matters: always take the topic lock before a subscription lock.

P7.7 Demonstration

- A "talker" publishes numbered messages on `chatter`; three "listeners" on different CPUs print what they receive.
- One listener is deliberately slow: it loses the oldest messages (show the drop counter) while the publisher and the other listeners continue at full speed.
- Two topics and two publishers at the same time, using `ps_spin` with several callbacks.

P7.8 References

1. *Publish–subscribe pattern*, Wikipedia.
2. ROS, *Writing a Simple Publisher and Subscriber (C++)*.
3. ROS 2 documentation, *Understanding topics*.
4. Linux manual page mq_overview(7) (POSIX message queues, for comparison).